

PE 文件导出表结构与 ROR13 哈希算法（知识点整理）

核心目标：掌握 PE 文件导出表的解析方法，理解 IMAGE_DOS_HEADER → IMAGE_NT_HEADERS → IMAGE_DATA_DIRECTORY →

IMAGE_EXPORT_DIRECTORY 的层层定位链路，学会 ROR13 哈希算法对函数名计算哈希值，并用哈希比对替代明文 GetProcAddress 调用。

- 涉及的核心结构体

(一) IMAGE_DOS_HEADER (DOS 头, PE 文件开头)

```
typedef struct _IMAGE_DOS_HEADER {  
    WORD    e_magic;        // 0x00 ← "MZ" (0x5A4D), 校验是否为有效 DOS 文件  
    WORD    e_cblp;         // 0x02  
    WORD    e_cp;           // 0x04  
    WORD    e_crlc;         // 0x06  
    WORD    e_cparhdr;      // 0x08  
    WORD    e_minalloc;     // 0x0A  
    WORD    e_maxalloc;     // 0x0C  
    WORD    e_ss;           // 0x0E  
    WORD    e_sp;           // 0x10  
    WORD    e_csum;         // 0x12  
    WORD    e_ip;           // 0x14  
    WORD    e_cs;           // 0x16  
    WORD    e_lfarlc;       // 0x18  
    WORD    e_ovno;         // 0x1A  
    WORD    e_res[4];       // 0x1C  
    WORD    e_oemid;        // 0x24  
    WORD    e_oeminfo;      // 0x26  
    WORD    e_res2[10];     // 0x28  
    LONG    e_lfanew;       // 0x3C ← 指向 NT 头的文件偏移量 (核心字段)  
} IMAGE_DOS_HEADER;
```

核心字段：e_magic (校验) 和 e_lfanew (跳转 NT 头)。

(二) IMAGE_FILE_HEADER (PE 文件头)

```
typedef struct _IMAGE_FILE_HEADER {  
    WORD    Machine;        // 0x00 CPU 架构 (0x8664=x64, 0x14C=x86)  
    WORD    NumberOfSections; // 0x02 节区数量  
    DWORD   TimeDateStamp;   // 0x04 编译时间戳  
    DWORD   PointerToSymbolTable; // 0x08  
    DWORD   NumberOfSymbols; // 0x0C  
    WORD    SizeOfOptionalHeader; // 0x10 可选头大小  
    WORD    Characteristics; // 0x12 文件属性  
} IMAGE_FILE_HEADER;
```

(三) IMAGE_OPTIONAL_HEADER64 (64 位可选头) 关键字段

```
typedef struct _IMAGE_OPTIONAL_HEADER64 {  
    WORD    Magic;                                // 0x00    0x20B=PE32+  
    BYTE    MajorLinkerVersion;                   // 0x02  
    BYTE    MinorLinkerVersion;                   // 0x03  
    DWORD   SizeOfCode;                            // 0x04  
    DWORD   SizeOfInitializedData;                 // 0x08  
    DWORD   SizeOfUninitializedData;               // 0x0C  
    DWORD   AddressOfEntryPoint;                   // 0x10    入口点 RVA  
    DWORD   BaseOfCode;                            // 0x14  
    ULONGLONG ImageBase;                           // 0x18    默认加载基址  
    DWORD   SectionAlignment;                      // 0x20    节对齐  
    DWORD   FileAlignment;                         // 0x24    文件对齐  
    // ... 省略中间字段  
    DWORD   NumberOfRvaAndSizes;                   // 0x6C    数据目录数量  
    IMAGE_DATA_DIRECTORY DataDirectory[16];        // 0x70    ← 数据目录数组 (核心)  
} IMAGE_OPTIONAL_HEADER64;
```

关键偏移：DataDirectory[0] (导出表目录) 位于可选头偏移 $0x70 + 0x08 = 0x78$ 处？

实际计算：DataDirectory 在结构体中的偏移是 0x70，DataDirectory[0] 的 VirtualAddress 在 0x70，Size 在 0x74。但 ntHeaders + 0x88 定位到的是 DataDirectory[0].VirtualAddress，这是因为 ntHeaders 指向的是 IMAGE_NT_HEADERS 的起始 (Signature 处)

所以 $0x88 = 0x04$ (Signature) + $0x14$ (FileHeader) + $0x70$ (DataDirectory 偏移)。

(四) IMAGE_DATA_DIRECTORY (数据目录项)

```
typedef struct _IMAGE_DATA_DIRECTORY {  
    DWORD VirtualAddress;    // 0x00    数据目录的 RVA (相对虚拟地址)  
    DWORD Size;              // 0x04    数据目录的大小  
} IMAGE_DATA_DIRECTORY;
```

索引含义：DataDirectory[0] = 导出表，DataDirectory[1] = 导入表，DataDirectory[5] = 重定位表。

(五) IMAGE_EXPORT_DIRECTORY (导出表核心结构)

```
typedef struct _IMAGE_EXPORT_DIRECTORY {  
    DWORD Characteristics;    // 0x00    未使用，恒为 0  
    DWORD TimeDateStamp;      // 0x04    时间戳  
    WORD   MajorVersion;      // 0x08  
    WORD   MinorVersion;      // 0x0A  
    DWORD Name;                // 0x0C    模块名称的 RVA (如"kernel32.dll")  
    DWORD Base;                // 0x10    起始序号  
    DWORD NumberOfFunctions;   // 0x14    导出函数总数 (含无名函数)
```

```
    DWORD NumberOfNames;           // 0x18  ← 按名称导出的函数数量
    DWORD AddressOfFunctions;       // 0x1C  ← 函数地址表 RVA
    DWORD AddressOfNames;           // 0x20  ← 函数名称表 RVA
    DWORD AddressOfNameOrdinals;    // 0x24  ← 函数序号表 RVA
} IMAGE_EXPORT_DIRECTORY;
```

关键字段：NumberOfNames、AddressOfFunctions、AddressOfNames、AddressOfNameOrdinals 四个字段是遍历导出表的核心。

(六) IMAGE_NT_HEADERS64 (NT 头完整结构)

```
typedef struct _IMAGE_NT_HEADERS64 {
    DWORD Signature;                // 0x00  "PE\0\0" (0x00004550)
    IMAGE_FILE_HEADER FileHeader;    // 0x04
    IMAGE_OPTIONAL_HEADER64 OptionalHeader; // 0x18
} IMAGE_NT_HEADERS64;
```

● 涉及的关键宏与常量

(一) IMAGE_DOS_SIGNATURE (0x5A4D)

- 含义：DOS 头的有效标识，ASCII 字符为 "MZ"。
- 用途：校验文件是否为有效的 PE 文件。

(二) IMAGE_NT_SIGNATURE (0x00004550)

- 含义：NT 头的有效标识，ASCII 字符为 "PE\0\0"。
- 用途：在 DOS 头之后校验，确认是 Windows PE 可执行文件。

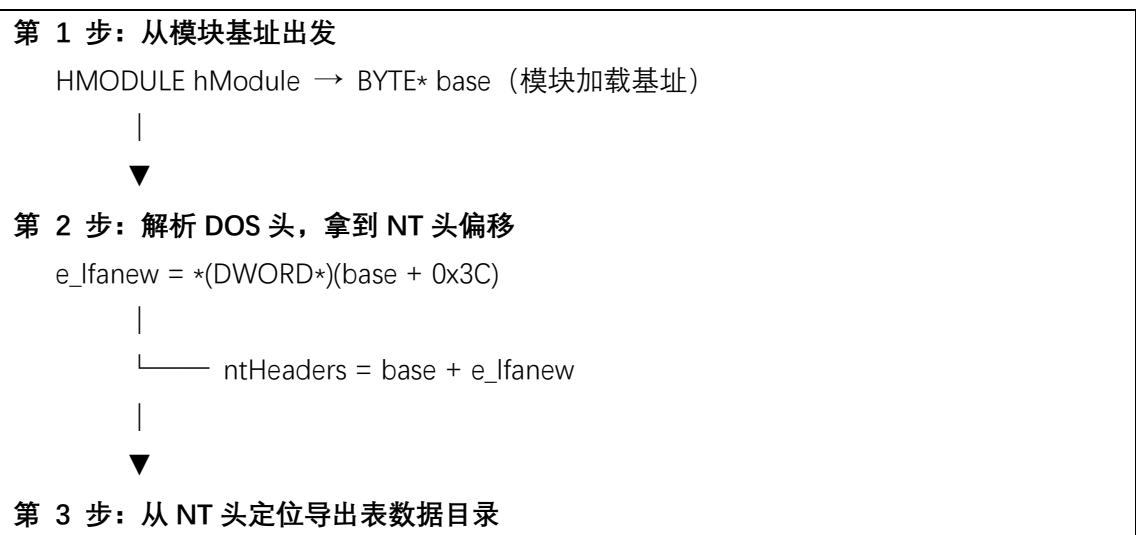
(三) IMAGE_DIRECTORY_ENTRY_EXPORT (值为 0)

- 含义：数据目录数组中的导出表索引。
- 用途：DataDirectory[IMAGE_DIRECTORY_ENTRY_EXPORT] 即为导出表目录项。

(四) IMAGE_SIZEOF_SHORT_NAME (值为 8)

- 含义：PE 导出/导入函数名的最大长度。
- 用途：某些 API 名称超过 8 字节需特别注意。

● 完整的利用链 (PE 结构层层定位图)



```
DataDirectory[0].VirtualAddress = *(DWORD*)(ntHeaders + 0x88)
```

|

└── 这个 RVA 就是导出表相对于模块基址的偏移

|



第 4 步：定位导出表结构

```
ExportTable = (IMAGE_EXPORT_DIRECTORY*)(base + exportRVA)
```

|



第 5 步：从导出表读取四个关键字段

	NumberOfNames	=	ExportTable + 0x18	
	AddressOfFunctions	=	ExportTable + 0x1C	
	AddressOfNames	=	ExportTable + 0x20	
	AddressOfNameOrdinals	=	ExportTable + 0x24	

|



第 6 步：构建三个数组指针（RVA → 实际地址）

```
nameArray = (DWORD*)(base + AddressOfNames)
```

```
ordinalArray = (WORD*)(base + AddressOfNameOrdinals)
```

```
funcArray = (DWORD*)(base + AddressOfFunctions)
```

|



第 7 步：遍历名称数组，ROR13 哈希比对

```
for (i = 0; i < NumberOfNames; i++) {  
    funcName = (char*)(base + nameArray[i]);  
    if (Ror13Hash(funcName) == targetHash) {  
        ordinal = ordinalArray[i];    // 拿到序号索引  
        funcRVA = funcArray[ordinal]; // 通过序号拿函数 RVA  
        return base + funcRVA;        // 返回函数绝对地址  
    }  
}
```

文字总结：

- DOS 头 → 读 e_lfanew（偏移 0x3C）→ 跳转到 NT 头。
- NT 头 → 读 DataDirectory[0].VirtualAddress（偏移 0x88）→ 拿到导出表 RVA。
- 导出表 → 读四个关键字段（NumberOfNames、AddressOfFunctions、AddressOfNames、AddressOfNameOrdinals）。

- 三数组 → 名称数组、序号数组、地址数组。
- 遍历匹配 → 对每个函数名算 ROR13 哈希，与目标哈希比对。
- 命中返回 → 通过序号数组找到函数 RVA，加上基址返回绝对地址。

● 核心偏移量速查表 (Windows 10/11 x64)

描述	表达式	偏移值 (HEX)
e_lfanew 在 DOS 头中的位置	DOS + 0x3C	0x3C
Signature 在 NT 头中的位置	NT + 0x00	0x00
FileHeader 在 NT 头中的位置	NT + 0x04	0x04
OptionalHeader 在 NT 头中的位置	NT + 0x18	0x18
DataDirectory[0].VirtualAddress (导出表 RVA)	NT + 0x88	0x88
NumberOfNames 在导出表中的位置	Export + 0x18	0x18
AddressOfFunctions 在导出表中的位置	Export + 0x1C	0x1C
AddressOfNames 在导出表中的位置	Export + 0x20	0x20
AddressOfNameOrdinals 在导出表中的位置	Export + 0x24	0x24

0x88 的计算: Signature(4) + FileHeader(20) + DataDirectory 偏移(0x70) = 0x04 + 0x14 + 0x70 = 0x88。

● ROR13 哈希算法详解

算法原理

```
DWORD Ror13Hash(const char* str) {
    DWORD hash = 0;
    while (*str) {
        hash = (hash >> 13) | (hash << (32 - 13)); // 循环右移 13 位
        hash += (unsigned char)*str;               // 加上当前字符
        str++;
    }
    return hash;
}
```

为什么用 ROR13

- 规避静态查杀：二进制文件中不出现 "CreateRemoteThread" 明文字符串。
- 轻量高效：纯位运算，无需额外库。
- 唯一性足够：32 位哈希空间，碰撞概率低。

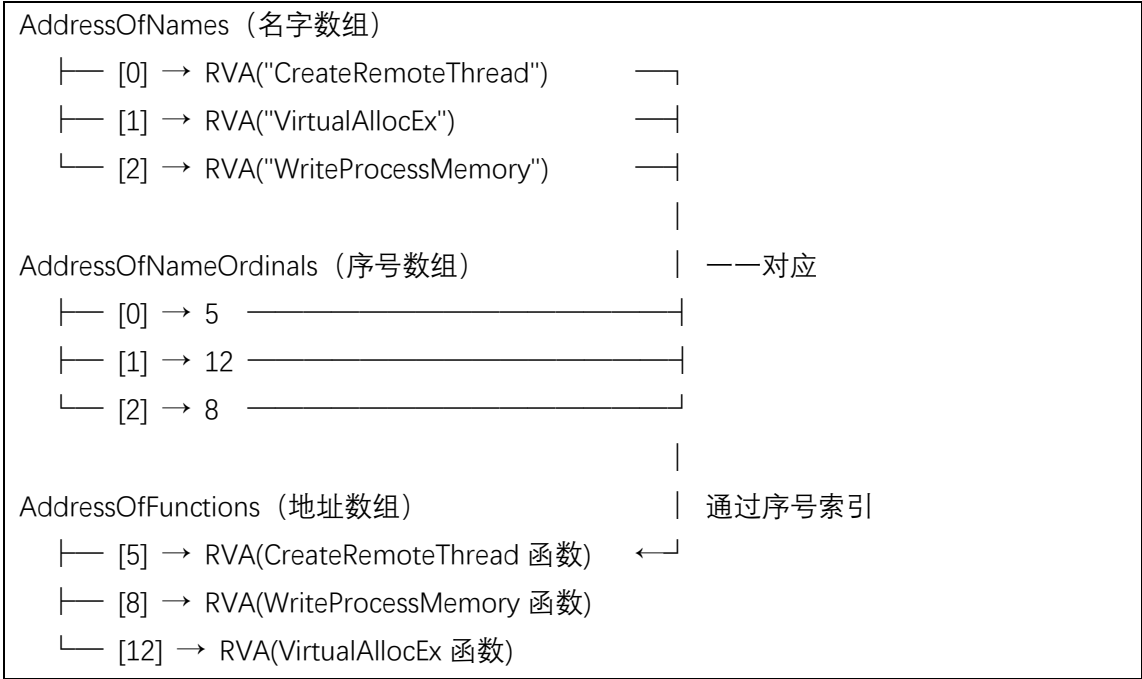
预计算哈希值

```
DWORD targetHash = Ror13Hash("CreateRemoteThread");
// 输出: 0x???????? (具体值由算法决定)
```

注意：ROR13 哈希值可以离线预计算，硬编码进代码，避免运行时计算字符串。

● 导出表三数组结构详解

(一) 三数组的关系



(二) 三数组的职责

数组	类型	作用
AddressOfNames	DWORD*	存储函数名字符串的 RVA
AddressOfNameOrdinals	WORD*	存储函数在地址数组中的索引
AddressOfFunctions	DWORD*	存储函数地址的 RVA

(三) 遍历逻辑

```
for (DWORD i = 0; i < NumberOfNames; i++) {  
    char* funcName = (char*)(base + nameArray[i]);    // 第 i 个函数名  
    if (Ror13Hash(funcName) == targetHash) {  
        WORD ordinal = ordinalArray[i];                // 该函数在地址数组中的索引  
        DWORD funcRVA = funcArray[ordinal];            // 函数 RVA  
        return base + funcRVA;                          // 绝对地址  
    }  
}
```

● 其他必要的扩展知识点

(一) 为什么要用哈希替代明文字符串？

维度	明文 GetProcAddress	ROR13 哈希比对
IAT 痕迹	出现 GetProcAddress 和函数名	无任何敏感字符串
静态查杀	杀软直接匹配字符串	只看到一个 DWORD 常量
隐蔽性	★	★★★★

(二) e_lfanew 为什么是 0x3C？

- DOS 头早期设计时，e_lfanew 被放在偏移 0x3C 处（DOS 头总大小 0x40 字节，e_lfanew 是倒数第 4 个字段）。
- 这是 PE 格式的历史遗留设计，微软为了保证向后兼容，一直没有改动。

(三) 导出表 RVA 与实际地址的转换

- PE 文件在硬盘上是“未拉伸”的（文件对齐），在内存中是“拉伸”的（节对齐）。
- `DataDirectory[0].VirtualAddress` 是 RVA（Relative Virtual Address），加上模块基址 `base` 才是绝对虚拟地址。
- 关键：`GetModuleHandle` 返回的是模块**加载到内存后**的基址，所以 `base + RVA` 直接得到内存中的实际地址。

(四) 为什么用 `GetModuleHandleA` 而不用 `GetModuleHandleW`?

- 代码中模块名是 `"kernel32.dll"`（窄字符），所以用 A 版本。
- 若使用 `L"kernel32.dll"`（宽字符），则需用 W 版本。
- 两者功能完全一致，只是字符编码不同。

(五) `IMAGE_EXPORT_DIRECTORY` 中的 `Name` 字段

- 指向模块名称字符串的 RVA（如 `"kernel32.dll"`）。
- 用途：确认该导出表属于哪个模块（调试用，遍历时不需要）。

(六) 导出表的边界问题

- `NumberOfFunctions` 可能大于 `NumberOfNames`（存在无名函数，通过序号导出）。
- 本课只遍历 `NumberOfNames`，只处理按名称导出的函数。
- 若需处理无名函数，需遍历 `NumberOfFunctions` 并通过 `Base` 字段计算序号。